DATA STRUCTURES ASSIGNMENT

Name: Loshini V

Register Number: 1965260053

Course Name: Data Structures

Course Code: CSA0304

1. Define Data Structure

Definition

A data structure is a way of organizing, storing and managing data in a computer so that it can be accessed and modified efficiently. It defines how data items are arranged and what operations can be performed on them.

Need for Data Structures

- They organize large amounts of data systematically.
- They make searching, sorting, insertion, deletion and updating easier.
- They can improve program performance and help use memory efficiently.
- They help represent real-world information and relationships.

They also provide a clear plan for how a program stores and processes information. Choosing a suitable data structure can reduce unnecessary work and make a program easier to understand, test and maintain.

Characteristics of a Good Data Structure

A good data structure stores information in a suitable form, supports the required operations, uses memory carefully and gives acceptable performance as the amount of data grows. The best choice depends on the problem and the operations performed most often.

Types of Data Structures

Types of Data Structures

Primitive data structures: basic data types such as int, char, float and double.

Non-primitive data structures: structures built from basic types.

Linear structures arrange elements in sequence. Examples include arrays, linked lists, stacks and queues.

Non-linear structures arrange elements hierarchically or as networks. Examples include trees and graphs.

Common Examples

An array stores items in indexed positions; a linked list connects items using links; a stack follows Last In, First Out (LIFO); a queue follows First In, First Out (FIFO); a tree represents parent-child relationships; and a graph represents connections between items.

Choosing the Right Structure

For example, an array is useful when many elements must be accessed by position. A queue is suitable for tasks waiting in order, while a graph is useful for finding routes between places. Selecting the right structure improves clarity and efficiency.

Basic Operations and Applications

Common operations include traversal, searching, insertion, deletion, sorting and merging. Arrays store student marks, stacks support undo operations, queues manage print jobs, trees represent folders, and graphs model road or computer networks.

Everyday Applications

Data structures are used in student-record systems, online shopping, banking, navigation apps, operating systems and social networks. For example, a navigation app can represent locations as vertices and roads as edges in a graph, then search for a suitable route.

Conclusion

Data structures are important because they organize data and make programs easier to develop, maintain and run efficiently.

2. C Program for Matrix Addition, Subtraction and MultiplicationAim

To write a C program that performs matrix addition, subtraction and multiplication using two-dimensional arrays.

Logic

Addition and subtraction combine elements at matching row and column positions. Matrix multiplication calculates each result element by multiplying a row of the first matrix by a column of the second matrix and adding the products.

Conditions for Matrix Operations

For addition and subtraction, both matrices must have the same number of rows and columns. For multiplication, the number of columns in the first matrix must equal the number of rows in the second matrix. If these conditions are not satisfied, that operation cannot be performed on the given matrices.

Algorithm

1. Read the row and column sizes of both matrices.
2. Read all elements of matrix A and matrix B.
3. If their dimensions match, add and subtract corresponding elements.
4. If the columns of A equal the rows of B, initialise each product element to zero and accumulate A[i][k] * B[k][j].
5. Display each valid result and print a message for any operation that is not possible.

C Program

```
#include <stdio.h>

int main(void)
{
    int A[10][10], B[10][10];
    int Sum[10][10], Diff[10][10], Product[10][10];
    int r1, c1, r2, c2, i, j, k;

    printf("Enter rows and columns of A: ");
    scanf("%d %d", &r1, &c1);
    printf("Enter rows and columns of B: ");
    scanf("%d %d", &r2, &c2);

    if (r1 < 1 || r1 > 10 || c1 < 1 || c1 > 10 ||
        r2 < 1 || r2 > 10 || c2 < 1 || c2 > 10) {
        printf("Invalid size. Use dimensions from 1 to 10.\\n");
        return 1;
    }

    printf("Enter elements of A:\\n");
```

```
for (i = 0; i < r1; i++)
    for (j = 0; j < c1; j++)
        scanf("%d", &A[i][j]);

printf("Enter elements of B:\\n");
for (i = 0; i < r2; i++)
    for (j = 0; j < c2; j++)
        scanf("%d", &B[i][j]);

if (r1 == r2 && c1 == c2) {
    for (i = 0; i < r1; i++)
        for (j = 0; j < c1; j++) {
            Sum[i][j] = A[i][j] + B[i][j];
            Diff[i][j] = A[i][j] - B[i][j];
        }

    printf("\\nAddition (A + B):\\n");
    for (i = 0; i < r1; i++) {
        for (j = 0; j < c1; j++)
            printf("%d\\t", Sum[i][j]);
        printf("\\n");
    }

    printf("\\nSubtraction (A - B):\\n");
    for (i = 0; i < r1; i++) {
        for (j = 0; j < c1; j++)
            printf("%d\\t", Diff[i][j]);
        printf("\\n");
    }
} else {
    printf("\\nAddition/subtraction not possible: dimensions differ.\\n");
}

if (c1 == r2) {
    for (i = 0; i < r1; i++)
        for (j = 0; j < c2; j++) {
            Product[i][j] = 0;
            for (k = 0; k < c1; k++)
                Product[i][j] += A[i][k] * B[k][j];
        }

    printf("\\nMultiplication (A x B):\\n");
    for (i = 0; i < r1; i++) {
        for (j = 0; j < c2; j++)
            printf("%d\\t", Product[i][j]);
        printf("\\n");
    }
} else {
    printf("\\nMultiplication not possible: columns of A must equal rows of B.\\n");
}
```

```
    return 0;
}
```

Sample Input

Matrix A (2 x 2):
1 2
3 4

Matrix B (2 x 2):
5 6
7 8

Sample Output

Addition (A + B):
6 8
10 12

Subtraction (A - B):
-4 -4
-4 -4

Multiplication (A x B):
19 22
43 50

Explanation

Addition and subtraction require both matrices to have equal dimensions. Multiplication requires the number of columns in matrix A to equal the number of rows in matrix B. The program checks these conditions before calculating the results.

Understanding the Sample

For addition, the first elements are added: 1 + 5 = 6. For subtraction, they are subtracted: 1 - 5 = -4. For multiplication, the first result is (1 × 5) + (2 × 7) = 19. The same row-by-column method is repeated for every element in the product matrix.

Why Two-Dimensional Arrays Are Useful

A two-dimensional array stores values in rows and columns, which matches the structure of a matrix. Nested loops allow the program to visit each element systematically. The program also checks dimensions before performing operations, helping avoid invalid calculations.

Time Complexity

For matrices with n rows and n columns, addition and subtraction take $O(n^2)$ time because each element is processed once. Standard matrix multiplication takes $O(n^3)$ time because it uses three nested loops. The example program limits each matrix dimension to 10 to keep the storage bounded.

Result

Thus, a C program was written to perform matrix addition, subtraction and multiplication using two-dimensional arrays.

3. Explain the Role of Arrays in Memory Management

Introduction

An array is a collection of elements of the same data type stored in consecutive memory locations. Each element is accessed by an index, and in C the index starts at zero.

How Arrays Use Memory

When an array is declared, memory is reserved for its elements. Since the elements are stored contiguously, the address of an element can be calculated from the base address, its index and the size of each element.

For example, if an integer occupies 4 bytes and the base address is 1000, the address of A[3] is 1000 + (3 × 4) = 1012, assuming zero-based indexing and the usual contiguous layout. This predictable calculation is one reason indexed array access is fast.

Address of A[i] = Base address + (i x size of each element)

In C, two-dimensional arrays are stored in row-major order: the first row is stored first, followed by the second row, and so on.

Role of Arrays in Memory Management

1. Contiguous storage

Array elements occupy adjacent memory locations, making their layout predictable.

2. Fast indexed access

An element can usually be accessed in O(1) time using its index.

3. Organized storage

One array stores many related values, such as marks, prices or sensor readings.

4. Cache locality

Sequential processing of nearby elements often makes good use of the processor cache.

5. Tables and matrices

Two-dimensional arrays store grids of values and support matrix calculations.

6. Fixed-size allocation

A fixed-size array reserves a known amount of memory. An oversized array may waste memory, while an undersized array may not hold all required data.

7. Dynamic memory

Dynamic allocation is useful when the number of items is not known until the program runs. In C, the programmer should check whether malloc() or calloc() returned a valid pointer, use the allocated memory within its limits and call free() when finished. This helps prevent memory leaks.

When the size is needed at runtime, malloc() or calloc() can allocate a block of memory. The program should release it using free() when it is no longer needed.

Example in C

```
#include <stdio.h>

int main(void)
{
    int marks[5] = {78, 85, 90, 67, 88};
    int i;

    for (i = 0; i < 5; i++)
        printf("marks[%d] = %d\\n", i, marks[i]);

    printf("Array size = %zu bytes\\n", sizeof(marks));
    return 0;
}
```

Advantages

- Simple storage of multiple values of the same type.
- Direct access using an index.
- Useful for loops, sorting, searching and matrix operations.
- Contiguous layout can improve sequential processing

8. Predictable memory layout

Because array elements are stored one after another, the memory needed for an array can be estimated from the number of elements and the size of each element. For example, an array of 5 integers uses 5 × sizeof(int) bytes.

9. Support for algorithms

Arrays provide the basic storage used by many algorithms, including linear search, binary search on sorted data, sorting methods and matrix calculations. They are also used to build other structures such as heaps and hash-table buckets.

Limitations

- Fixed-size arrays have limited capacity.
- Insertion or deletion in the middle may require shifting elements.
- C does not automatically check array index bounds.
- An array holds elements of one declared type.

Practical Example

A college may store the marks of 60 students in an integer array. A loop can calculate the total, average, highest mark and lowest mark without creating a separate variable for every student. This makes the program shorter and easier to update.

Conclusion

Arrays organize related data in contiguous memory and provide fast indexed access. They are useful for calculations and data processing, but programmers must choose suitable sizes and avoid invalid index access.

In conclusion, arrays are an important part of memory organisation in C. Their predictable layout and quick access are useful, but safe programs must respect array bounds and manage dynamically allocated memory carefully.

Overall Conclusion

Data structures organize data for efficient processing. Matrix operations demonstrate the use of two-dimensional arrays in C, while arrays illustrate how contiguous memory storage supports fast and systematic access.